

SYNC FIFO

Bugs & Solution

Before bug:

```
23
24 always @(posedge clk or negedge rst_n) begin
25     if (!rst_n) begin
26         wr_ptr <= 0;
27     end
28     if (wr_ptr <= FIFO_DEPTH) begin
```

After sloving bug

```
5 /*.....*/
7 always @(posedge f_if.clk or negedge f_if.rst_n) begin
8     if (!f_if.rst_n) begin
9         wr_ptr <= 0;
10
11
12         f_if.overflow <= 0;
13         f_if.wr_ack <= 0;
14     end
```

reseting wr_ack & over_flow flags

Before bug:

```
41
42 always @(posedge clk or negedge rst_n) begin
43     if (!rst_n) begin
44         rd_ptr <= 0;
45     end
```

After sloving bug:

```
38
39 always @(posedge f_if.clk or negedge f_if.rst_n) begin
40
41     if (!f_if.rst_n) begin
42         rd_ptr <= 0;
43         f_if.data_out <= 0;
44         f_if.underflow <= 0;
45     end
```

reseting data_out & under_flow flag

Before bug:

```
65 assign empty = (count == 0)? 1 : 0;  
66 assign underflow = (empty && rd_en)? 1 : 0;
```

After sloving bug:

```
38  
39 always @(posedge f_if.clk or negedge f_if.rst_n) begin  
40  
41     if (!f_if.rst_n) begin  
42         rd_ptr <= 0;  
43         f_if.data_out <= 0;  
44         f_if.underflow <= 0;  
45     end  
46     else if (f_if.rd_en && count != 0) begin  
47         f_if.data_out <= mem[rd_ptr];  
48         rd_ptr <= rd_ptr + 1;  
49     end  
50     else begin  
51         if (f_if.empty && f_if.rd_en)  
52             f_if.underflow <= 1;  
53         else  
54             f_if.underflow <= 0;  
55     end  
56 end  
57
```

Registering the under_flow flag

Before bug:

```
6 assign underflow = (empty && rd_en)? 1 : 0;  
7 assign almostfull = (count == FIFO_DEPTH-2)? 1 : 0;  
8 assign almostempty = (count == 1)? 1 : 0;
```

After sloving bug:

```
75 assign f_if.empty = (count == 0)? 1 : 0;  
76 assign f_if.almostfull = (count == f_if.FIFO_DEPTH - 1)? 1 : 0;  
77 assign f_if.almostempty = (count == 1)? 1 : 0;
```

Before bug:

```

55     end
56     else begin
57         if ( ({wr_en, rd_en} == 2'b10) && !full)
58             count <= count + 1;
59         else if ( ({wr_en, rd_en} == 2'b01) && !empty)
60             count <= count - 1;
61     end
62 end

```

After sloving bug:

```

57
58     always @(posedge f_if.clk or negedge f_if.rst_n) begin
59         if (!f_if.rst_n) begin
60             count <= 0;
61         end
62         else begin
63             if ({f_if.wr_en, f_if.rd_en} == 2'b10) && !f_if.full)
64                 count <= count + 1;
65             else if ({f_if.wr_en, f_if.rd_en} == 2'b11) && f_if.empty)
66                 count <= count + 1;
67             else if ({f_if.wr_en, f_if.rd_en} == 2'b01) && !f_if.empty)
68                 count <= count - 1;
69             else if ({f_if.wr_en, f_if.rd_en} == 2'b11) && f_if.full)
70                 count <= count - 1;
71         end
72     end

```

Handling the cases when wr_en =1 & rd_en=1

Complete design:

```

1  module FIFO(fifo_if,DUT f_if);
2
3      localparam max_fifo_addr = $clog2(f_if.FIFO_DEPTH);
4      reg [f_if.FIFO_WIDTH-1:0] mem [f_if.FIFO_DEPTH-1:0];
5      reg [max_fifo_addr-1:0] wr_ptr=0;
6      reg [max_fifo_addr-1:0] rd_ptr=0;
7      reg [max_fifo_addr:0] count=0;
8
9
10     /*.....*/
11     always @(posedge f_if.clk or negedge f_if.rst_n) begin
12         if (!f_if.rst_n) begin
13             wr_ptr <= 0;
14
15             f_if.overflow <= 0;
16             f_if.wr_ack <= 0;
17         end
18         else if (f_if.wr_en && count < f_if.FIFO_DEPTH) begin
19             mem[wr_ptr] <= f_if.data_in;
20             f_if.wr_ack <= 1;
21             wr_ptr <= wr_ptr + 1;
22         end
23         else begin
24             f_if.wr_ack <= 0;
25             if (f_if.full && f_if.wr_en)
26                 f_if.overflow <= 1;
27             else
28                 f_if.overflow <= 0;
29         end
30     end
31 end
32
33     always @(posedge f_if.clk or negedge f_if.rst_n) begin
34
35         if (!f_if.rst_n) begin
36             rd_ptr <= 0;
37             f_if.data_out <= 0;
38             f_if.underflow <= 0;
39         end

```

```

40     else if (f_if.rd_en && count != 0) begin
41         f_if.data_out <= mem[rd_ptr];
42         rd_ptr <= rd_ptr + 1;
43     end
44     else begin
45         if (f_if.empty && f_if.rd_en)
46             f_if.underflow <= 1;
47         else
48             f_if.underflow <= 0;
49     end
50 end
51
52 always @(posedge f_if.clk or negedge f_if.rst_n) begin
53     if (!f_if.rst_n) begin
54         count <= 0;
55     end
56     else begin
57         if ({f_if.wr_en, f_if.rd_en} == 2'b10) && !f_if.full)
58             count <= count + 1;
59         else if ({f_if.wr_en, f_if.rd_en} == 2'b11) && f_if.empty)
60             count <= count + 1;
61         else if ({f_if.wr_en, f_if.rd_en} == 2'b01) && !f_if.empty)
62             count <= count - 1;
63         else if ({f_if.wr_en, f_if.rd_en} == 2'b11) && f_if.full)
64             count <= count - 1;
65     end
66 end
67
68 assign f_if.full      = (count == f_if.FIFO_DEPTH) ? 1 : 0;
69 assign f_if.empty     = (count == 0) ? 1 : 0;
70 assign f_if.almostfull = (count == f_if.FIFO_DEPTH - 1) ? 1 : 0;
71 assign f_if.almostempty = (count == 1) ? 1 : 0;
72
73 /*.....assertions.....*/
74
75 /*.....reset assertion.....*/
76 always_comb begin
77     if (f_if.rst_n == 0)
78         o 1: assert final (count == 0 && rd_ptr == 0 && wr_ptr == 0 && f_if.underflow==0 && f_if.overflow==0
79             && f_if.wr_ack==0 && f_if.full==0 && f_if.empty==1);
80 end
81
82 a1 : assert property (@(posedge f_if.clk) disable iff (!f_if.rst_n)
83     (f_if.wr_en && !f_if.full) |>= f_if.wr_ack);
84 c1 : cover property (@(posedge f_if.clk) disable iff (!f_if.rst_n)
85     (f_if.wr_en && !f_if.full) |>= f_if.wr_ack);
86
87 a2 : assert property (@(posedge f_if.clk) disable iff (!f_if.rst_n)
88     (f_if.wr_en && f_if.full) |>= f_if.overflow);
89 c2 : cover property (@(posedge f_if.clk) disable iff (!f_if.rst_n)
90     (f_if.wr_en && f_if.full) |>= f_if.overflow);
91
92 a3 : assert property (@(posedge f_if.clk) disable iff (!f_if.rst_n)
93     (f_if.rd_en && f_if.empty) |>= f_if.underflow);
94 c3 : cover property (@(posedge f_if.clk) disable iff (!f_if.rst_n)
95     (f_if.rd_en && f_if.empty) |>= f_if.underflow);
96
97 a4 : assert property (@(posedge f_if.clk) disable iff (!f_if.rst_n)
98     (!count) |>= f_if.empty);
99 c4 : cover property (@(posedge f_if.clk) disable iff (!f_if.rst_n)
100    (!count) |>= f_if.empty);
101
102 a5 : assert property (@(posedge f_if.clk) disable iff (!f_if.rst_n)
103    (count == f_if.FIFO_DEPTH) |>= f_if.full);
104 c5 : cover property (@(posedge f_if.clk) disable iff (!f_if.rst_n)
105    (count == f_if.FIFO_DEPTH) |>= f_if.full);
106
107 a6 : assert property (@(posedge f_if.clk) disable iff (!f_if.rst_n)
108    (count == f_if.FIFO_DEPTH - 1) |>= f_if.almostfull);
109 c6 : cover property (@(posedge f_if.clk) disable iff (!f_if.rst_n)
110    (count == f_if.FIFO_DEPTH - 1) |>= f_if.almostfull);
111
112 a7 : assert property (@(posedge f_if.clk) disable iff (!f_if.rst_n)

```



```

112 a7 : assert property (@(posedge f_if.clk) disable iff (!f_if.rst_n)
113                      (count == 1) |-> f_if.almostempty);
114 c7 : cover property (@(posedge f_if.clk) disable iff (!f_if.rst_n)
115                      (count == 1) |-> f_if.almostempty);
116
117 a8 : assert property (@(posedge f_if.clk) disable iff (!f_if.rst_n)
118                      (f_if.wr_en && wr_ptr == 7&&!f_if.full) |=> (wr_ptr == 0));
119 c8 : cover property (@(posedge f_if.clk) disable iff (!f_if.rst_n)
120                      (f_if.wr_en && wr_ptr == 7&&!f_if.full) |=> (wr_ptr == 0));
121
122 a9 : assert property (@(posedge f_if.clk) disable iff (!f_if.rst_n)
123                      (f_if.rd_en && rd_ptr == 7&&!f_if.empty) |=> (rd_ptr == 0));
124 c9 : cover property (@(posedge f_if.clk) disable iff (!f_if.rst_n)
125                      (f_if.rd_en && rd_ptr == 7&&!f_if.empty) |=> (rd_ptr == 0));
126 `endif
127
128 endmodule
129

```

Top:

```

1 module fifo_top();
2   bit clk;
3   initial begin
4     clk=0;
5     forever #1 clk=~clk;
6   end
7   fifo_if f_if (clk);
8   FIFO dut (f_if);
9   FIFO_tb tb(f_if);
10  monitor mon (f_if);
11  /*.....reset assertion.....*/
12  always_comb begin
13    if (f_if.rst_n == 0)
14      o_1: assert final (dut.count == 0 && dut.rd_ptr == 0 && dut.wr_ptr == 0
15      &&f_if.underflow==0&&f_if.overflow==0&&f_if.wr_ack==0&&f_if.full==0&&f_if.empty==1);
16  end
17
18 endmodule

```

If:

```

1 interface fifo_if (clk);
2   input bit clk;
3   /*.....parameters.....*/
4   parameter FIFO_WIDTH = 16;
5   parameter FIFO_DEPTH = 8;
6   /*.....inputs and outputs.....*/
7   logic [FIFO_WIDTH-1:0] data_in;
8   logic rst_n, wr_en, rd_en;
9   logic [FIFO_WIDTH-1:0] data_out;
10  logic wr_ack, overflow;
11  logic full, empty, almostfull, almostempty, underflow;
12  modport DUT (input clk,rst_n,wr_en,rd_en,data_in,output data_out,wr_ack,overflow,full,empty,almostfull,almostempty,underflow);
13  modport TEST (output rst_n,wr_en,rd_en,data_in,input data_out,wr_ack,overflow,full,empty,almostfull,almostempty,underflow,clk );
14  modport MONITOR (input clk,rst_n,wr_en,rd_en,data_in, data_out,wr_ack,overflow,full,empty,almostfull,almostempty,underflow);
15
16 endinterface

```

Shared pkg:

```
1  package shared_pkg;
2      /*.....parameters.....*/
3  parameter FIFO_WIDTH = 16;
4  parameter FIFO_DEPTH = 8;
5  parameter ADDR_SIZE=3;
6      bit test_finished;
7      event ev;
8  integer error_count=0;
9  integer correct_count=0;
10 endpackage
```

Scoreboard:

```
1
2  package FIFO_scoreboard_pkg;
3
4      import FIFO_transaction_pkg::*;
5      import shared_pkg::*;
6
7  class FIFO_scoreboard;
8
9      // ----- Signals -----
10     reg [FIFO_WIDTH-1:0] data_out_ref;
11     reg [FIFO_WIDTH-1:0] mem_ref [$];
12
13     // ----- Reference Model -----
14     function void reference_model(FIFO_transaction F);
15     if(!F.rst_n) begin
16         mem_ref.delete();
17         data_out_ref = 0;
18     end
19     else begin
20         if (F.wr_en && F.rd_en) begin
21             if(mem_ref.size() == 0)
22                 mem_ref.push_back(F.data_in);
23             else if (mem_ref.size() == FIFO_DEPTH)
24                 data_out_ref = mem_ref.pop_front();
25             else begin
26                 data_out_ref = mem_ref.pop_front();
27                 mem_ref.push_back(F.data_in);
28             end
29         end
30         else if (F.wr_en && !F.rd_en) begin
31             if (mem_ref.size() != FIFO_DEPTH)
32                 mem_ref.push_back(F.data_in);
33         end
34         else if (!F.wr_en && F.rd_en) begin
35             if (mem_ref.size() != 0)
36                 data_out_ref = mem_ref.pop_front();
37         end
38     end
39 endfunction
```

```

40
41
42 // ----- Check Data -----
43 task check_data(FIFO_transaction f);
44     reference_model(f);
45
46     if (f.data_out != data_out_ref) begin
47         error_count++;
48         $display(".....ERROR.....");
49         $display("data_in: %b | wr_en: %b | rd_en: %b", f.data_in, f.wr_en, f.rd_en);
50         $display("data_out=%b data_out_ref=%b", f.data_out, data_out_ref);
51         $display("time : %t ", $time);
52     end
53     else begin
54         correct_count++;
55         $display(".....PASSED.....");
56         $display(" data_in=%b wr_en=%b rd_en=%b", f.data_in, f.wr_en, f.rd_en);
57         $display("data_out=%b data_out_ref=%b", f.data_out, data_out_ref);
58     end
59 endtask
60
61 endclass
62
63 endpackage
64

```

Coverage:

```

1
2 package FIFO_coverage_pkg;
3 import FIFO_transaction_pkg::*;
4
5 class FIFO_coverage;
6     FIFO_transaction F_cvg_txn;
7
8     covergroup fifo_cg;
9         WR_EN: coverpoint F_cvg_txn.wr_en { bins zero = {0}; bins one = {1}; }
10        RD_EN: coverpoint F_cvg_txn.rd_en { bins zero = {0}; bins one = {1}; }
11        WR_ACK: coverpoint F_cvg_txn.wr_ack { bins zero = {0}; bins one = {1}; }
12        FULL: coverpoint F_cvg_txn.full { bins zero = {0}; bins one = {1}; }
13        OVER_FLOW: coverpoint F_cvg_txn.overflow { bins zero = {0}; bins one = {1}; }
14        EMPTY: coverpoint F_cvg_txn.empty { bins zero = {0}; bins one = {1}; }
15        ALMOST_EMPTY: coverpoint F_cvg_txn.almostempty { bins zero = {0}; bins one = {1}; }
16        ALMOST_FULL: coverpoint F_cvg_txn.almostfull { bins zero = {0}; bins one = {1}; }
17        UNDER_FLOW: coverpoint F_cvg_txn.underflow { bins zero = {0}; bins one = {1}; }
18        //===== CROSS COVERAGES =====//
19        c1 : cross WR_EN, RD_EN, WR_ACK {
20            bins zero_zero_zero = binsof(WR_EN.zero) && binsof(RD_EN.zero) && binsof(WR_ACK.zero);
21            illegal_bins zero_zero_one = binsof(WR_EN.zero) && binsof(RD_EN.zero) && binsof(WR_ACK.one);
22            bins zero_one_zero = binsof(WR_EN.zero) && binsof(RD_EN.one) && binsof(WR_ACK.zero);
23            illegal_bins zero_one_one = binsof(WR_EN.zero) && binsof(RD_EN.one) && binsof(WR_ACK.one);
24            bins one_zero_zero = binsof(WR_EN.one) && binsof(RD_EN.zero) && binsof(WR_ACK.zero);
25            bins one_zero_one = binsof(WR_EN.one) && binsof(RD_EN.zero) && binsof(WR_ACK.one);
26            bins one_one_zero = binsof(WR_EN.one) && binsof(RD_EN.one) && binsof(WR_ACK.zero);
27            bins one_one_one = binsof(WR_EN.one) && binsof(RD_EN.one) && binsof(WR_ACK.one);
28            option.cross_auto_bin_max = 0;
29        }
30
31        c2 : cross WR_EN, RD_EN, FULL {
32            bins zero_zero_zero = binsof(WR_EN.zero) && binsof(RD_EN.zero) && binsof(FULL.zero);
33            bins zero_zero_one = binsof(WR_EN.zero) && binsof(RD_EN.zero) && binsof(FULL.one);
34            bins zero_one_zero = binsof(WR_EN.zero) && binsof(RD_EN.one) && binsof(FULL.zero);
35            illegal_bins zero_one_one = binsof(WR_EN.zero) && binsof(RD_EN.one) && binsof(FULL.one);
36            bins one_zero_zero = binsof(WR_EN.one) && binsof(RD_EN.zero) && binsof(FULL.zero);
37            bins one_zero_one = binsof(WR_EN.one) && binsof(RD_EN.zero) && binsof(FULL.one);
38            bins one_one_zero = binsof(WR_EN.one) && binsof(RD_EN.one) && binsof(FULL.zero);
39            illegal_bins one_one_one = binsof(WR_EN.one) && binsof(RD_EN.one) && binsof(FULL.one);
40            option.cross_auto_bin_max = 0;

```



```

41 }
42
43 c3 : cross WR_EN, RD_EN, EMPTY {
44     bins zero_zero_zero = binsof(WR_EN.zero) && binsof(RD_EN.zero) && binsof(EMPTY.zero);
45     bins zero_zero_one = binsof(WR_EN.zero) && binsof(RD_EN.zero) && binsof(EMPTY.one);
46     bins zero_one_zero = binsof(WR_EN.zero) && binsof(RD_EN.one) && binsof(EMPTY.zero);
47     bins zero_one_one = binsof(WR_EN.zero) && binsof(RD_EN.one) && binsof(EMPTY.one);
48     bins one_zero_zero = binsof(WR_EN.one) && binsof(RD_EN.zero) && binsof(EMPTY.zero);
49     bins one_zero_one = binsof(WR_EN.one) && binsof(RD_EN.zero) && binsof(EMPTY.one);
50     bins one_one_zero = binsof(WR_EN.one) && binsof(RD_EN.one) && binsof(EMPTY.zero);
51     bins one_one_one = binsof(WR_EN.one) && binsof(RD_EN.one) && binsof(EMPTY.one);
52     option.cross_auto_bin_max = 0;
53 }
54
55 c4 : cross WR_EN, RD_EN, OVER_FLOW {
56     bins zero_zero_zero = binsof(WR_EN.zero) && binsof(RD_EN.zero) && binsof(OVER_FLOW.zero);
57     illegal_bins zero_zero_one = binsof(WR_EN.zero) && binsof(RD_EN.zero) && binsof(OVER_FLOW.one);
58     bins zero_one_zero = binsof(WR_EN.zero) && binsof(RD_EN.one) && binsof(OVER_FLOW.zero);
59     illegal_bins zero_one_one = binsof(WR_EN.zero) && binsof(RD_EN.one) && binsof(OVER_FLOW.one);
60     bins one_zero_zero = binsof(WR_EN.one) && binsof(RD_EN.zero) && binsof(OVER_FLOW.zero);
61     bins one_zero_one = binsof(WR_EN.one) && binsof(RD_EN.zero) && binsof(OVER_FLOW.one);
62     bins one_one_zero = binsof(WR_EN.one) && binsof(RD_EN.one) && binsof(OVER_FLOW.zero);
63     bins one_one_one = binsof(WR_EN.one) && binsof(RD_EN.one) && binsof(OVER_FLOW.one);
64     option.cross_auto_bin_max = 0;
65 }
66
67 c5 : cross WR_EN, RD_EN, UNDER_FLOW {
68     bins zero_zero_zero = binsof(WR_EN.zero) && binsof(RD_EN.zero) && binsof(UNDER_FLOW.zero);
69     illegal_bins zero_zero_one = binsof(WR_EN.zero) && binsof(RD_EN.zero) && binsof(UNDER_FLOW.one);
70     bins zero_one_zero = binsof(WR_EN.zero) && binsof(RD_EN.one) && binsof(UNDER_FLOW.zero);
71     bins zero_one_one = binsof(WR_EN.zero) && binsof(RD_EN.one) && binsof(UNDER_FLOW.one);
72     bins one_zero_zero = binsof(WR_EN.one) && binsof(RD_EN.zero) && binsof(UNDER_FLOW.zero);
73     illegal_bins one_zero_one = binsof(WR_EN.one) && binsof(RD_EN.zero) && binsof(UNDER_FLOW.one);
74     bins one_one_zero = binsof(WR_EN.one) && binsof(RD_EN.one) && binsof(UNDER_FLOW.zero);
75     bins one_one_one = binsof(WR_EN.one) && binsof(RD_EN.one) && binsof(UNDER_FLOW.one);
76     option.cross_auto_bin_max = 0;
77 }
78
79 c6 : cross WR_EN, RD_EN, ALMOST_FULL {
80     bins zero_zero_zero = binsof(WR_EN.zero) && binsof(RD_EN.zero) && binsof(ALMOST_FULL.zero);
81
82     c6 : cross WR_EN, RD_EN, ALMOST_FULL {
83         bins zero_zero_zero = binsof(WR_EN.zero) && binsof(RD_EN.zero) && binsof(ALMOST_FULL.zero);
84         bins zero_zero_one = binsof(WR_EN.zero) && binsof(RD_EN.zero) && binsof(ALMOST_FULL.one);
85         bins zero_one_zero = binsof(WR_EN.zero) && binsof(RD_EN.one) && binsof(ALMOST_FULL.zero);
86         bins zero_one_one = binsof(WR_EN.zero) && binsof(RD_EN.one) && binsof(ALMOST_FULL.one);
87         bins one_zero_zero = binsof(WR_EN.one) && binsof(RD_EN.zero) && binsof(ALMOST_FULL.zero);
88         bins one_zero_one = binsof(WR_EN.one) && binsof(RD_EN.zero) && binsof(ALMOST_FULL.one);
89         bins one_one_zero = binsof(WR_EN.one) && binsof(RD_EN.one) && binsof(ALMOST_FULL.zero);
90         bins one_one_one = binsof(WR_EN.one) && binsof(RD_EN.one) && binsof(ALMOST_FULL.one);
91         option.cross_auto_bin_max = 0;
92     }
93
94     c7 : cross WR_EN, RD_EN, ALMOST_EMPTY {
95         bins zero_zero_zero = binsof(WR_EN.zero) && binsof(RD_EN.zero) && binsof(ALMOST_EMPTY.zero);
96         bins zero_zero_one = binsof(WR_EN.zero) && binsof(RD_EN.zero) && binsof(ALMOST_EMPTY.one);
97         bins zero_one_zero = binsof(WR_EN.zero) && binsof(RD_EN.one) && binsof(ALMOST_EMPTY.zero);
98         bins zero_one_one = binsof(WR_EN.zero) && binsof(RD_EN.one) && binsof(ALMOST_EMPTY.one);
99         bins one_zero_zero = binsof(WR_EN.one) && binsof(RD_EN.zero) && binsof(ALMOST_EMPTY.zero);
100        bins one_zero_one = binsof(WR_EN.one) && binsof(RD_EN.zero) && binsof(ALMOST_EMPTY.one);
101        bins one_one_zero = binsof(WR_EN.one) && binsof(RD_EN.one) && binsof(ALMOST_EMPTY.zero);
102        bins one_one_one = binsof(WR_EN.one) && binsof(RD_EN.one) && binsof(ALMOST_EMPTY.one);
103        option.cross_auto_bin_max = 0;
104    }
105
106 endgroup : fifo_cg
107
108 function new();
109     fifo_cg = new();
110     F_cvgtxn = new();
111 endfunction
112
113 function void sampled_data(FIFO_transaction F_txn);
114     F_cvgtxn = F_txn;
115     fifo_cg.sample();
116 endfunction
117
118 endclass : FIFO_coverage
119
120 endpackage : FIFO_coverage_pkg

```

Src list:

```
1  shared_pkg.sv
2  FIFO.sv
3  fifo_if.sv
4  FIFO_transaction_pkg.sv
5  FIFO_scoreboard_pkg.sv
6  FIFO_coverage_pkg.sv
7  FIFO_tb.sv
8  monitor.sv
9  fifo_top.sv
10
```

Monitor:

```
1  import FIFO_transaction_pkg::*;
2  import FIFO_scoreboard_pkg::*;
3  import FIFO_coverage_pkg::*;
4  import shared_pkg::*;
5  //////////////////////////////////////////////////
6  module monitor (fifo_if.MONITOR f_if);
7      FIFO_scoreboard scr_b=new();
8      FIFO_coverage cov=new();
9      FIFO_transaction trans=new();
10     initial begin
11         forever begin
12             @ ev;
13             @( negedge f_if.clk);
14             trans.data_in=f_if.data_in;
15             trans.rst_n=f_if.rst_n;
16             trans.wr_en=f_if.wr_en;
17             trans.rd_en=f_if.rd_en;
18             trans.data_out=f_if.data_out;
19             trans.wr_ack=f_if.wr_ack;
20             trans.overflow=f_if.overflow;
21             trans.full=f_if.full;
22             trans.empty=f_if.empty;
23             trans.almostfull=f_if.almostfull;
24             trans.almostempty=f_if.almostempty;
25             trans.underflow=f_if.underflow;
26             fork
27                 begin
28                     cov.sampled_data(trans);
29                 end
30                 begin
31                     scr_b.check_data(trans);
32                 end
33             join
34             if(test_finished) begin
35                 $display("error count value is : %d & correct count value is : %d",error_count,correct_count);
36                 $stop;
37             end
38         end
39     end
40 endmodule
```

Tb:

```
1  import FIFO_transaction_pkg::*;
2  import shared_pkg::*;
3  //////////////////////////////////
4  module FIFO_tb(fifo_if.TEST f_if);
5  //////////////////////////////////
6  /*.....assert reset task .....*/
7  task assert_reset;
8      f_if.rst_n=1;
9      @(negedge f_if.clk);
10     -> ev;
11     f_if.rst_n=0;
12     @(negedge f_if.clk);
13     -> ev;
14     //@( negedge f_if.clk);
15     f_if.rst_n=1;
16     //@( negedge f_if.clk);
17 endtask
18 integer i;
19 initial begin
20
21     FIFO_transaction fifo_tb;
22     fifo_tb=new();
23     assert_reset();
24
25     for (i=0;i<20000;i=i+1) begin
26         assert(fifo_tb.randomize());
27         f_if.rst_n=fifo_tb.rst_n;
28         f_if.data_in=fifo_tb.data_in;
29         f_if.rd_en=fifo_tb.rd_en;
30         f_if.wr_en=fifo_tb.wr_en;
31         @( negedge f_if.clk);
32         -> ev;
33         //@( negedge f_if.clk);
34     end
35     test_finished=1;
36
37 end
38
39 endmodule
40
```

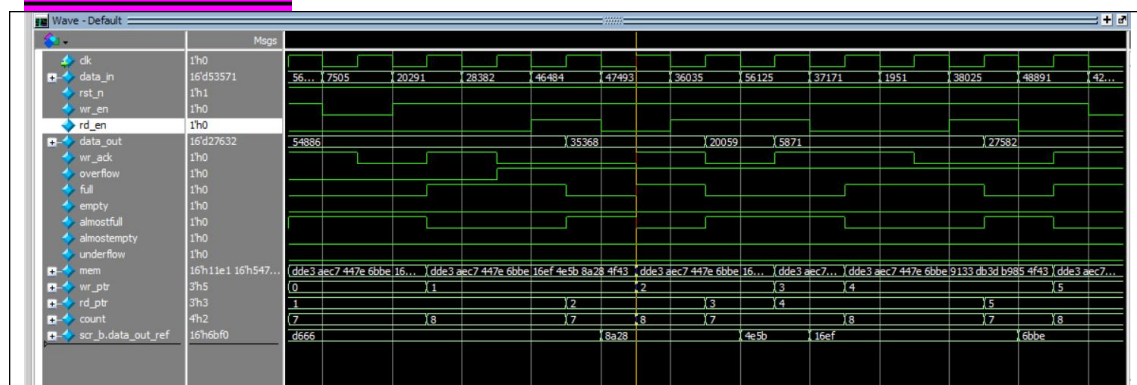
Do file:

```
1  vlib work
2  vlog -f src_files.list.txt +define+SIM +cover -covercells
3
4  vsim -voptargs=+acc work.fifo_top -classdebug -uvmcontrol=all -cover
5  #add wave /fifo_top/f_if/*
6  run 0
7  do wave.do
8  coverage save fifo_top.ucdb -onexit
9  run -all
10
11
```

Verification plan :

Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check (Assertions)
Reset must clear FIFO (wr_ptr=0, rd_ptr=0, count=0)	Apply async reset (rst_n=0 → 1)	Coverpoint for reset sequence	reset
Write acknowledge when wr_en=1 and FIFO not full	Drive wr_en=1 while full=0	Write_Acknowledge_cover	Write_Acknowledge
Overflow flag when wr_en=1 and FIFO full	Fill FIFO to full, then drive wr_en=1	Write_overflow_cover	Write_overflow
Underflow flag when rd_en=1 and FIFO empty	Empty FIFO, then drive rd_en=1	Write_Underflow_cover	Write_Underflow
Empty flag asserted when count=0	Drain FIFO completely	Empty_cover	Empty
Full flag asserted when count=FIFO_DEPTH	Fill FIFO completely	full_cover	full
Almost full flag when count=FIFO_DEPTH-1	Drive writes until 1 slot remains	almostfull_cover	almostfull
Almost empty flag when count=1	Drain FIFO until 1 element remains	almostempty_cover	almostempty
Write pointer wraps at max address	Continuously write until pointer rolls over	Pointer_Wraparound_W_cover	Pointer_Wraparound_W
Read pointer wraps at max address	Continuously read until pointer rolls over	Pointer_Wraparound_R_cover	Pointer_Wraparound_R

Waveform:



Output :


```

# .....PASSED.....
# data_in=1101000101000011 wr_en=0 rd_en=0
# data_out=0110101111110000 data_out_ref=0110101111110000
# error count value is : 0 & correct count value is : 20002
# ** Note: $atom : monitor.sv(38)

```

Coverage texts:

```

37
38 =====
39 === Instance: /fifo_top/dut
40 === Design Unit: work.FIFO
41 =====
42
43 Assertion Coverage:
44   Assertions              10      10      0  100.00%
45 -----
46 Name                      File(Line)          Failure    Pass
47                               Count              Count
48 -----
49 /fifo_top/dut/o_1         FIFO.sv(84)              0          1
50 /fifo_top/dut/a1          FIFO.sv(88)              0          1
51 /fifo_top/dut/a2          FIFO.sv(93)              0          1
52 /fifo_top/dut/a3          FIFO.sv(98)              0          1
53 /fifo_top/dut/a4          FIFO.sv(103)             0          1
54 /fifo_top/dut/a5          FIFO.sv(108)             0          1
55 /fifo_top/dut/a6          FIFO.sv(113)             0          1
56 /fifo_top/dut/a7          FIFO.sv(118)             0          1
57 /fifo_top/dut/a8          FIFO.sv(123)             0          1
58 /fifo_top/dut/a9          FIFO.sv(128)             0          1
59 Branch Coverage:
60 -----
61 Directive Coverage:
62   Directives              9      9      0  100.00%
63 -----
64 DIRECTIVE COVERAGE:
65 -----
66 Name                      Design Design  Lang File(Line)    Hits Status
67                               Unit  UnitType
68 -----
69 /fifo_top/dut/c1          FIFO    Verilog  SVA  FIFO.sv(90)     7113 Covered
70 /fifo_top/dut/c2          FIFO    Verilog  SVA  FIFO.sv(95)     6705 Covered
71 /fifo_top/dut/c3          FIFO    Verilog  SVA  FIFO.sv(100)    105 Covered
72 /fifo_top/dut/c4          FIFO    Verilog  SVA  FIFO.sv(105)    338 Covered
73 /fifo_top/dut/c5          FIFO    Verilog  SVA  FIFO.sv(110)   9627 Covered
74 /fifo_top/dut/c6          FIFO    Verilog  SVA  FIFO.sv(115)   5972 Covered
75 /fifo_top/dut/c7          FIFO    Verilog  SVA  FIFO.sv(120)    480 Covered
76 /fifo_top/dut/c8          FIFO    Verilog  SVA  FIFO.sv(125)    807 Covered
77 /fifo_top/dut/c9          FIFO    Verilog  SVA  FIFO.sv(130)    643 Covered
78
79 Statement Coverage:
80   Enabled Coverage      Bins    Hits    Misses Coverage
81 -----
82   Statements            29      29      0  100.00%
83
84 =====Statement Details=====

```


389	Statement Coverage:				
390	Enabled Coverage	Bins	Hits	Misses	Coverage
391	-----	----	----	-----	-----
392	Statements	29	29	0	100.00%
393					
394	=====Statement Details=====				
395					
396	Statement Coverage for instance /fifo_top/dut --				
397					
398	Line	Item	Count	Source	
399	----	----	-----	-----	
400	File FIFO.sv				
401	8			module FIFO(fifo_if.DUT f_if);	
402					
403	9				
404					
405	10			localparam max_fifo_addr = \$clog2(f_if.FIFO_DEPTH);	
406					
407	11			reg [f_if.FIFO_WIDTH-1:0] mem [f_if.FIFO_DEPTH-1:0];	
408					
409	12			reg [max_fifo_addr-1:0] wr_ptr=0;	
410					
411	13			reg [max_fifo_addr-1:0] rd_ptr=0;	
412					
413	14			reg [max_fifo_addr:0] count=0;	
414					
415	15				
416					
417	16			/*.....*/	
418					
419	17	1	20197	always @(posedge f_if.clk or negedge f_if.rst_n) begin	
420					
421	18			if (!f_if.rst_n) begin	
422					
423	19	1	391	wr_ptr <= 0;	
424					
425	20				
426					
427	21				
428					
429	22	1	391	f_if.overflow <= 0;	
430					
431	23	1	391	f_if.wr_ack <= 0;	
432					
433	24			end	
59	Branch Coverage:				
60	Enabled Coverage	Bins	Hits	Misses	Coverage
61	-----	----	----	-----	-----
62	Branches	27	27	0	100.00%
63					
64	=====Branch Details=====				
65					
66	Branch Coverage for instance /fifo_top/dut				
67					
68	Line	Item	Count	Source	
69	----	----	-----	-----	
70	File FIFO.sv				
71	-----IF Branch-----				
72	18		20197	Count coming in to IF	
73	18	1	391	if (!f_if.rst_n) begin	
74					
75	25	1	7168	else if (f_if.wr_en && count < f_if.FIFO_DEPTH) begin	
76					
77	30	1	12638	else begin	
78					
79	Branch totals: 3 hits of 3 branches = 100.00%				
80					
81	-----IF Branch-----				
82	32		12638	Count coming in to IF	
83	32	1	6780	if (f_if.full && f_if.wr_en)	
84					
85	34	1	5858	else	
86					
87	Branch totals: 2 hits of 2 branches = 100.00%				
88					
89	-----IF Branch-----				
90	41		20197	Count coming in to IF	
91	41	1	391	if (!f_if.rst_n) begin	
92					
93	46	1	5826	else if (f_if.rd_en && count != 0) begin	
94					
95	50	1	13980	else begin	
96					
97	Branch totals: 3 hits of 3 branches = 100.00%				

```

3 =====
4 === Instance: /fifo_top/f_if
5 === Design Unit: work.fifo_if
6 =====
7 Toggle Coverage:
8   Enabled Coverage      Bins      Hits      Misses      Coverage
9   -----
10  Toggles                86       86        0     100.00%
11
12 =====Toggle Details=====
13
14 Toggle Coverage for instance /fifo_top/f_if --
15
16   Node      1H->0L      0L->1H      "Coverage"
17   -----
18   almostempty      1        1     100.00
19   almostfull       1        1     100.00
20   clk              1        1     100.00
21   data_in[15-0]    1        1     100.00
22   data_out[15-0]   1        1     100.00
23   empty            1        1     100.00
24   full             1        1     100.00
25   overflow         1        1     100.00
26   rd_en            1        1     100.00
27   rst_n            1        1     100.00
28   underflow        1        1     100.00
29   wr_ack           1        1     100.00
30   wr_en            1        1     100.00
31
32 Total Node Count      =      43
33 Toggled Node Count    =      43
34 Untoggled Node Count  =       0
35
36 Toggle Coverage      =    100.00% (86 of 86 bins)
37

```

```

930 === Design Unit: work.FIFO_coverage_pkg
931 =====
932
933 Covergroup Coverage:
934   Covergroups          1      na      na     100.00%
935   Coverpoints/Crosses  16      na      na
936   Covergroup Bins      66      66      0     100.00%
937
938 Covergroup              Metric      Goal      Bins      Status
939 -----
940
941 TYPE /FIFO_coverage_pkg/FIFO_coverage/fifo_cg      100.00%      100      -      Covered
942 covered/total bins:      66      66      -
943 missing/total bins:      0      66      -
944 % Hit:      100.00%      100      -
945 Coverpoint WR_EN      100.00%      100      -      Covered
946 covered/total bins:      2      2      -
947 missing/total bins:      0      2      -
948 % Hit:      100.00%      100      -
949 bin zero      5913      1      -      Covered
950 bin one      14088      1      -      Covered
951 Coverpoint RD_EN      100.00%      100      -      Covered
952 covered/total bins:      2      2      -
953 missing/total bins:      0      2      -
954 % Hit:      100.00%      100      -
955 bin zero      14001      1      -      Covered
956 bin one      6000      1      -      Covered
957 Coverpoint WR_ACK      100.00%      100      -      Covered
958 covered/total bins:      2      2      -
959 missing/total bins:      0      2      -
960 % Hit:      100.00%      100      -
961 bin zero      12834      1      -      Covered
962 bin one      7168      1      -      Covered
963 Coverpoint FULL      100.00%      100      -      Covered
964 covered/total bins:      2      2      -
965 missing/total bins:      0      2      -
966 % Hit:      100.00%      100      -
967 bin zero      10280      1      -      Covered
968 bin one      9722      1      -      Covered
969 Coverpoint OVER_FLOW      100.00%      100      -      Covered
970 covered/total bins:      2      2      -
971 missing/total bins:      0      2      -
972 % Hit:      100.00%      100      -
973 bin zero      11391      1      -      Covered
974 bin one      8611      1      -      Covered
975 Coverpoint EMPTY      100.00%      100      -      Covered
976 covered/total bins:      2      2      -
977 missing/total bins:      0      2      -

```